



NETTOYAGE DE DONNÉES AVEC PYTHON

ENTREPRISE BOTTLENECK,
MARCHAND DE VIN TRÈS PRESTIGIEUX

1. Problématiques

- ▶ ERP et site web non connectés → double saisie, manque d'automatisation
- ▶ Gestion des stocks difficile → risques d'erreurs et d'écarts entre ERP et site.
- ▶ Visibilité limitée sur les ventes en ligne → peu d'accès aux données, analyses réduites.
- ▶ Processus artisanaux → perte de temps, manque de fiabilité.
- ▶ Pas de pilotage centralisé → vision globale de l'activité fragmentée

2. Plan d'action

- ▶ Consolidation des données "produits"
- ▶ Pas de problème d'encodage
- ▶ Tester l'unicité des clés
- ▶ Suppression ou modification des lignes justifiées
- ▶ Fusionner les exports ERP, web et table de liaison
- ▶ Vérification des jointures (type de jointure, nombre de produits jointés, etc.)
- ▶ Calcul du chiffre d'affaires : Calcul par produit et total en ligne.
- ▶ Analyse de la cohérence des prix
- ▶ Identifier, lister et représenter graphiquement les valeurs aberrantes.

3. Importation de la bibliothèque « pandas » et la librairie « numpy »

```
[1]: import pandas as pd  
import numpy as np
```

Chargement des données

```
[2]: df_erp = pd.read_excel("Fichier_erp.xlsx")  
df_liaison = pd.read_excel("fichier_liaison.xlsx")  
df_web = pd.read_excel("Fichier_web.xlsx")
```

4. Observation des données

```
[3]: df_erp.head()
```

```
[3]:
```

	product_id	onsale_web	price	stock_quantity	stock_status
0	3847	1	24.2	0	outofstock
1	3849	1	34.3	0	outofstock
2	3850	1	20.8	0	outofstock
3	4032	1	14.1	0	outofstock
4	4039	1	46.0	0	outofstock

```
[4]: df_liaison.head()
```

```
[4]:
```

	product_id	id_web
0	3847	15298
1	3849	15296
2	3850	15300
3	4032	19814
4	4039	19815

```
[5]: df_web.head()
```

```
[5]:
```

	sku	virtual	downloadable	rating_count	average_rating	total_s
0	16004	0	0	0	0.0	
1	NaN	0	0	0	NaN	
2	15075	0	0	0	0.0	

5. Données ERP

5.1. Analyse des données

```
[6]: df_erp.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 825 entries, 0 to 824
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	product_id	825 non-null	int64
1	onsale_web	825 non-null	int64
2	price	825 non-null	float64
3	stock_quantity	825 non-null	int64
4	stock_status	825 non-null	object

```
dtypes: float64(1), int64(3), object(1)
```

```
memory usage: 32.4+ KB
```

5. Données ERP

5.2. Vérification des valeurs manquantes

```
[7]: df_erp.isnull().sum()
```

```
[7]: product_id      0  
      onsale_web     0  
      price         0  
      stock_quantity 0  
      stock_status   0  
      dtype: int64
```


5. Données ERP

5.3. Vérification de la clé primaire

```
[8]: df_erp['product_id'].is_unique
```

```
[8]: True
```

5. Données ERP

5.4. Affichage des statistiques

```
[9]: df_erp.describe()
```

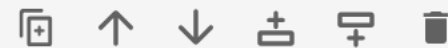
```
[9]:
```

	product_id	onsale_web	price	stock_quantity
count	825.000000	825.000000	825.000000	825.000000
mean	5162.597576	0.869091	32.384000	26.583030
std	902.644635	0.337506	26.832271	45.875948
min	3847.000000	0.000000	-8.000000	-1.000000
25%	4348.000000	1.000000	14.600000	1.000000
50%	4907.000000	1.000000	24.400000	11.000000
75%	5805.000000	1.000000	42.000000	34.000000
max	7338.000000	1.000000	225.000000	578.000000

5. Données ERP

5.5. Vérification des doublons

```
[10]: df_erp.duplicated().sum()
```



```
[10]: 0
```

5. Données ERP

5.6. Vérification de la corrélation entre stock_quantity et stock_status

```
[11]: df_erp["stock_status"].unique()
```

```
[11]: array(['outofstock', 'instock'], dtype=object)
```

```
[12]: df_erp[(df_erp["stock_status"] == "outofstock") & (df_erp["stock_quantity"] != 0)]
```

```
[12]:
```

	product_id	onsale_web	price	stock_quantity	stock_status
19	4055	0	86.1	1	outofstock
49	4090	0	73.0	6	outofstock
50	4092	0	47.0	6	outofstock
449	4973	0	10.0	-1	outofstock
573	5700	1	44.5	-1	outofstock
630	5826	1	41.2	12	outofstock
696	6125	0	14.2	48	outofstock

```
[13]: df_erp[(df_erp["stock_status"] == "instock") & (df_erp["stock_quantity"] == 0)]
```

```
[13]:
```

	product_id	onsale_web	price	stock_quantity	stock_status
443	4954	1	25.0	0	instock

5. Données ERP

5.7. Constats

- ▶ Aucune valeur manquante détectée dans les données ERP
- ▶ La colonne `product_id` est bien la clé primaire de ce dataframe
- ▶ Les types de variables sont corrects
- ▶ On a détecté quelques anomalies: prix négatif, stock quantité négatif
- ▶ Les incohérences ont été détectées sur la colonne `stock_status` qui doit être "instock" si le `stock_quantity` est différent de 0 et "outofstock" sinon

5. Données ERP

5.8. Correction des anomalies

```
[14]: colonnes_ciblees = ["stock_quantity", "price"] ### On remplace "le prix négatif et stock quantité négatif" en positif
df_erp[colonnes_ciblees] = df_erp[colonnes_ciblees].abs()
df_erp
```

```
[14]:
```

	product_id	onsale_web	price	stock_quantity	stock_status
0	3847	1	24.2	0	outofstock
1	3849	1	34.3	0	outofstock
2	3850	1	20.8	0	outofstock
3	4032	1	14.1	0	outofstock
4	4039	1	46.0	0	outofstock
...	...	...	...	...	...
820	7203	0	45.0	30	instock
821	7204	0	45.0	9	instock
822	7247	1	54.8	23	instock
823	7329	0	26.5	14	instock
824	7338	1	16.3	45	instock

825 rows × 5 columns

```
[15]: df_erp["stock_status"] = np.where(df_erp["stock_quantity"] == 0, "outofstock", "instock")
```

5. Données ERP

5.8. Correction des anomalies

5.8.1. Vérification

```
[16]: df_erp[(df_erp["stock_status"]=="outofstock") & (df_erp["stock_quantity"] != 0)]
```

```
[16]:  product_id  onsale_web  price  stock_quantity  stock_status
```

```
[17]: df_erp[(df_erp["stock_status"]=="instock") & (df_erp["stock_quantity"] == 0)]
```

```
[17]:  product_id  onsale_web  price  stock_quantity  stock_status
```

6. Données WEB

6.1. Analyse de données

```
[18]: df_web.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1513 entries, 0 to 1512
Data columns (total 28 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sku                    1428 non-null   object
1   virtual                1513 non-null   int64
2   downloadable            1513 non-null   int64
3   rating_count           1513 non-null   int64
4   average_rating          1430 non-null   float64
5   total_sales             1430 non-null   float64
6   tax_status              716 non-null    object
7   tax_class               0 non-null      float64
8   post_author            1430 non-null   float64
9   post_date              1430 non-null   datetime64[ns]
10  post_date_gmt           1430 non-null   datetime64[ns]
11  post_content            0 non-null      float64
12  post_title              1430 non-null   object
13  post_excerpt            716 non-null    object
14  post_status             1430 non-null   object
15  comment_status          1430 non-null   object
16  ping_status             1430 non-null   object
17  post_password           0 non-null      float64
18  post_name               1430 non-null   object
19  post_modified           1430 non-null   datetime64[ns]
20  post_modified_gmt       1430 non-null   datetime64[ns]
21  post_content_filtered    0 non-null      float64
22  post_parent             1430 non-null   float64
23  guid                    1430 non-null   object
24  menu_order              1430 non-null   float64
25  post_type               1430 non-null   object
26  post_mime_type           714 non-null    object
27  comment_count            1430 non-null   float64
dtypes: datetime64[ns](4), float64(10), int64(3), object(11)
memory usage: 331.1+ KB
```

6. Données WEB

6.2. Vérification des valeurs manquantes

```
[19]: df_web.isnull().sum()
```

```
[19]: sku                        85
      virtual                     0
      downloadable               0
      rating_count                0
      average_rating             83
      total_sales                83
      tax_status                 797
      tax_class                 1513
      post_author                83
      post_date                  83
      post_date_gmt              83
      post_content              1513
      post_title                 83
      post_excerpt               797
      post_status                83
      comment_status             83
      ping_status                83
      post_password              1513
      post_name                  83
      post_modified              83
      post_modified_gmt          83
      post_content_filtered      1513
      post_parent                83
      guid                      83
      menu_order                 83
      post_type                  83
      post_mime_type             799
      comment_count              83
      dtype: int64
```

6. Données WEB

6.3. Vérification de la clé primaire

```
[20]: df_web['sku'].is_unique
```

```
[20]: False
```


6. Données WEB

6.4. Vérification des doublons

```
[21]: df_web['sku'].duplicated().sum()
```

```
[21]: 798
```

6. Données WEB

6.5. Affichage des statistiques

```
[22]: df_web.describe()
```

```
[22]:
```

	virtual	downloadable	rating_count	average_rating	total_sales
count	1513.0	1513.0	1513.0	1430.0	1430.000000
mean	0.0	0.0	0.0	0.0	3.855245
min	0.0	0.0	0.0	0.0	0.000000
25%	0.0	0.0	0.0	0.0	0.000000
50%	0.0	0.0	0.0	0.0	1.000000
75%	0.0	0.0	0.0	0.0	4.000000
max	0.0	0.0	0.0	0.0	96.000000
std	0.0	0.0	0.0	0.0	7.702346

6. Données WEB

6.6. Constats

- ▶ Plusieurs valeurs manquantes détectées dans les données Web
- ▶ La colonne SKU ne répond pas en tant que clé primaire
- ▶ Certains Dtypes de variables sont incorrects
- ▶ Quelques anomalies détectées: présence de doublons liée à la colonne post_type qui enregistre pour un élément, son produit et son attachment

6. Données WEB

6.6. Constats

6.6.1. Conversion Colonne 'total_sales' en entier

```
[23]: df_web["total_sales"] = df_web["total_sales"].fillna(0).astype(int)
df_web.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1513 entries, 0 to 1512
Data columns (total 28 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   sku                                    1428 non-null   object
1   virtual                              1513 non-null   int64
2   downloadable                          1513 non-null   int64
3   rating_count                          1513 non-null   int64
4   average_rating                       1430 non-null   float64
5   total_sales                           1513 non-null   int64
6   tax_status                            716 non-null    object
7   tax_class                             0 non-null      float64
8   post_author                           1430 non-null   float64
9   post_date                             1430 non-null   datetime64[ns]
10  post_date_gmt                         1430 non-null   datetime64[ns]
11  post_content                           0 non-null      float64
12  post_title                            1430 non-null   object
13  post_excerpt                           716 non-null    object
14  post_status                           1430 non-null   object
15  comment_status                         1430 non-null   object
16  ping_status                           1430 non-null   object
17  post_password                          0 non-null      float64
18  post_name                             1430 non-null   object
19  post_modified                         1430 non-null   datetime64[ns]
20  post_modified_gmt                     1430 non-null   datetime64[ns]
21  post_content_filtered                 0 non-null      float64
22  post_parent                           1430 non-null   float64
23  guid                                  1430 non-null   object
24  menu_order                           1430 non-null   float64
25  post_type                             1430 non-null   object
26  post_mime_type                        714 non-null    object
27  comment_count                         1430 non-null   float64
dtypes: datetime64[ns](4), float64(9), int64(4), object(11)
memory usage: 331.1+ KB
```

6. Données WEB

6.6. Constats

6.6.2. Suppression des valeurs manquantes dans la colonne 'SKU'

```
[24]: df_web.dropna(axis = 0, subset = ['sku'], inplace = True)  
df_web.isnull().sum()
```

```
[24]: sku                0  
virtual                0  
downloadable          0  
rating_count          0  
average_rating        0  
total_sales           0  
tax_status            714  
tax_class            1428  
post_author           0  
post_date             0  
post_date_gmt         0  
post_content         1428  
post_title            0  
post_excerpt          714  
post_status           0  
comment_status        0  
ping_status           0  
post_password         1428  
post_name             0  
post_modified         0  
post_modified_gmt     0  
post_content_filtered 1428  
post_parent           0  
guid                 0  
menu_order            0  
post_type             0  
post_mime_type        714  
comment_count         0  
dtype: int64
```


6. Données WEB

6.6. Constats

6.6.3. Suppression des doublons 'sku'

```
[25]: df_web["post_type"].unique()
```

```
[25]: array(['attachment', 'product'], dtype=object)
```

```
[26]: df_web = df_web[df_web['post_type'] == 'product']
```

6. Données WEB

6.6. Constats

6.6.4. Vérification: Suppression des doublons 'sku'

```
[27]: df_web["post_type"].unique()
```

```
[27]: array(['product'], dtype=object)
```

```
[28]: df_web['sku'].is_unique
```

```
[28]: True
```

7. Données fichier Liaison

```
[29]: df_liaison.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 825 entries, 0 to 824  
Data columns (total 2 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   product_id  825 non-null    int64  
1   id_web      734 non-null    object  
dtypes: int64(1), object(1)  
memory usage: 13.0+ KB
```

```
[30]: df_liaison.isnull().sum()
```

```
[30]: product_id    0  
      id_web      91  
      dtype: int64
```

7. Données fichier Liaison

7.1. Constats

- ▶ La colonne sku doit être renommée à id_web pour s'aligner au dataframe liaison
- ▶ On constate 91 valeurs manquantes dans la colonne id_web, on peut dire que ce sont des produits non vendus ou la vente n'a pas été enregistrée.

7. Données fichier Liaison

7.2. Renommer le nom de la colonne "id_web" par "sku"

```
[31]: df_liaison.rename(index=str,columns={'id_web': 'sku'}, inplace = True)  
df_liaison.head()
```

```
[31]:
```

	product_id	sku
0	3847	15298
1	3849	15296
2	3850	15300
3	4032	19814
4	4039	19815

8. Jointure des 3 dataframes

8.1. Première jointure "ERP et Liaison"

```
[32]: df_1 = pd.merge(df_erp, df_liaison, on='product_id', how='outer', indicator=True)
      df_1.head()
```

```
[32]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	sku	_merge
0	3847	1	24.2	0	outofstock	15298	both
1	3849	1	34.3	0	outofstock	15296	both
2	3850	1	20.8	0	outofstock	15300	both
3	4032	1	14.1	0	outofstock	19814	both
4	4039	1	46.0	0	outofstock	19815	both

```
[33]: df_1["_merge"].unique()
```

```
[33]: ['both']
      Categories (3, object): ['left_only', 'right_only', 'both']
```

```
[34]: df_1.drop("_merge", axis=1, inplace=True)
      df_1.head()
```

```
[34]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	sku
0	3847	1	24.2	0	outofstock	15298
1	3849	1	34.3	0	outofstock	15296
2	3850	1	20.8	0	outofstock	15300
3	4032	1	14.1	0	outofstock	19814
4	4039	1	46.0	0	outofstock	19815

8. Jointure des 3 dataframes

8.2. Deuxième jointure "df_1 et le fichier_web"

```
[35]: df_final = pd.merge(df_1, df_web, on="sku", how="outer", indicator=True)
df_final.head()
```

```
[35]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	sku	virtual	downloadable	rating_count	average_
0	3847	1	24.2	0	outofstock	15298	0.0	0.0	0.0	
1	3849	1	34.3	0	outofstock	15296	0.0	0.0	0.0	
2	3850	1	20.8	0	outofstock	15300	0.0	0.0	0.0	
3	4032	1	14.1	0	outofstock	19814	0.0	0.0	0.0	
4	4039	1	46.0	0	outofstock	19815	0.0	0.0	0.0	

5 rows × 34 columns

8. Jointure des 3 dataframes

8.2. Deuxième jointure "df_1 et le fichier_web"

```
[36]: df_final["_merge"].unique()
```

```
[36]: ['both', 'left_only']  
Categories (3, object): ['left_only', 'right_only', 'both']
```

```
[37]: df_final[df_final["_merge"]=="left_only"]
```

```
[37]:
```

	product_id	onsale_web	price	stock_quantity	stock_status	sku	virtual	downloadable	rating_count	average_rating	...	...
19	4055	0	86.1	1	instock	NaN	NaN	NaN	NaN	NaN	...	...
20	4090	0	73.0	6	instock	NaN	NaN	NaN	NaN	NaN	...	...
21	4092	0	47.0	6	instock	NaN	NaN	NaN	NaN	NaN	...	...
22	4195	0	14.1	0	outofstock	NaN	NaN	NaN	NaN	NaN	...	...
23	4209	0	73.5	0	outofstock	NaN	NaN	NaN	NaN	NaN	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...
717	5954	0	18.8	0	outofstock	15609	NaN	NaN	NaN	NaN	...	...
718	5955	0	27.3	0	outofstock	14377	NaN	NaN	NaN	NaN	...	...
720	5957	0	39.0	0	outofstock	13577	NaN	NaN	NaN	NaN	...	...
743	6100	0	12.9	0	outofstock	15529	NaN	NaN	NaN	NaN	...	...
823	7329	0	26.5	14	instock	14680-1	NaN	NaN	NaN	NaN	...	...

111 rows x 34 columns

► Il existe 111 produits non vendus dans le web ou la vente n'a pas été enregistrée.

```
[38]: df_final = df_final[df_final["_merge"]=="both"]  
df_final.drop("_merge", axis=1, inplace=True)
```

8. Jointure des 3 dataframes

8.2. Deuxième jointure "df_1 et le fichier_web"

```
[39]: df_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 714 entries, 0 to 824
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            714 non-null    int64
1   onsale_web            714 non-null    int64
2   price                 714 non-null    float64
3   stock_quantity        714 non-null    int64
4   stock_status          714 non-null    object
5   sku                   714 non-null    object
6   virtual               714 non-null    float64
7   downloadable          714 non-null    float64
8   rating_count          714 non-null    float64
9   average_rating        714 non-null    float64
10  total_sales           714 non-null    float64
11  tax_status            714 non-null    object
12  tax_class             0 non-null      float64
13  post_author           714 non-null    float64
14  post_date             714 non-null    datetime64[ns]
15  post_date_gmt         714 non-null    datetime64[ns]
16  post_content          0 non-null      float64
17  post_title            714 non-null    object
18  post_excerpt          714 non-null    object
19  post_status           714 non-null    object
20  comment_status        714 non-null    object
21  ping_status           714 non-null    object
22  post_password         0 non-null      float64
23  post_name             714 non-null    object
24  post_modified         714 non-null    datetime64[ns]
25  post_modified_gmt     714 non-null    datetime64[ns]
26  post_content_filtered 0 non-null      float64
27  post_parent           714 non-null    float64
28  guid                  714 non-null    object
29  menu_order            714 non-null    float64
30  post_type             714 non-null    object
```

9. Calcul du chiffre d'affaires par produit

```
[41]: df_final["CA_produit"] = df_final["price"] * df_final["total_sales"]
      df_final
```

[41]:	product_id	onsale_web	price	stock_quantity	stock_status	sku	virtual	downloadable	rating_count	average_rating	...
	0	3847	1 24.2	0	outofstock	15298	0.0	0.0	0.0	0.0	...
	1	3849	1 34.3	0	outofstock	15296	0.0	0.0	0.0	0.0	...
	2	3850	1 20.8	0	outofstock	15300	0.0	0.0	0.0	0.0	...
	3	4032	1 14.1	0	outofstock	19814	0.0	0.0	0.0	0.0	...
	4	4039	1 46.0	0	outofstock	19815	0.0	0.0	0.0	0.0	...
	...	...	...	...	...	...	...	...	...	...	...
	819	6930	1 8.4	83	instock	16135	0.0	0.0	0.0	0.0	...
	820	7023	1 27.5	15	instock	15891	0.0	0.0	0.0	0.0	...
	821	7025	1 69.0	2	instock	15887	0.0	0.0	0.0	0.0	...
	822	7247	1 54.8	23	instock	13127-1	0.0	0.0	0.0	0.0	...
	824	7338	1 16.3	45	instock	16230	0.0	0.0	0.0	0.0	...

10. Calcul du chiffre d'affaires total

```
[42]: CA_total = df_final["CA_produit"].sum()  
      print(CA_total)
```

```
70568.6
```


11. Détection des outliers

11.1.Méthode de Calcul des bornes avec IQR (intervalle interquartile)

```
[43]: Q1 = df_final["price"].quantile(0.25)
      Q3 = df_final["price"].quantile(0.75)
      IQR = Q3 - Q1

      borne_inf = Q1 - 1.5 * IQR
      borne_sup = Q3 + 1.5 * IQR
      print(borne_inf, borne_sup)

-28.012500000000003 84.28750000000001
```

11. Détection des outliers

11.2. Filtre des potentielles valeurs aberrantes

```
[44]: Valeurs_aberrantes = df_final[(df_final["price"] < borne_inf) | (df_final["price"] > borne_sup)]

print("Liste des prix aberrants :")
print(Valeurs_aberrantes["price"].values)

pourcentage_Valeurs_aberrantes = round(len(Valeurs_aberrantes) / len(df_final) * 100, 2)

print(f"Le nombre de valeurs manquantes est :{len(Valeurs_aberrantes)}, soit {pourcentage_Valeurs_aberrantes}%")
```

Liste des prix aberrants :

```
[100.  88.4 225. 126.5 85.6 176. 108.5 157. 104. 109.6 102.3 137.
 217.5 105. 105. 112.  86.8 92. 124.8 175. 191.3 93. 122. 114.
 135. 105.6 116.4 115. 121. 99. 115. 121. ]
```

Le nombre de valeurs manquantes est :32, soit 4.48%

11. Détection des outliers

11.3. Création de la boîte à moustaches

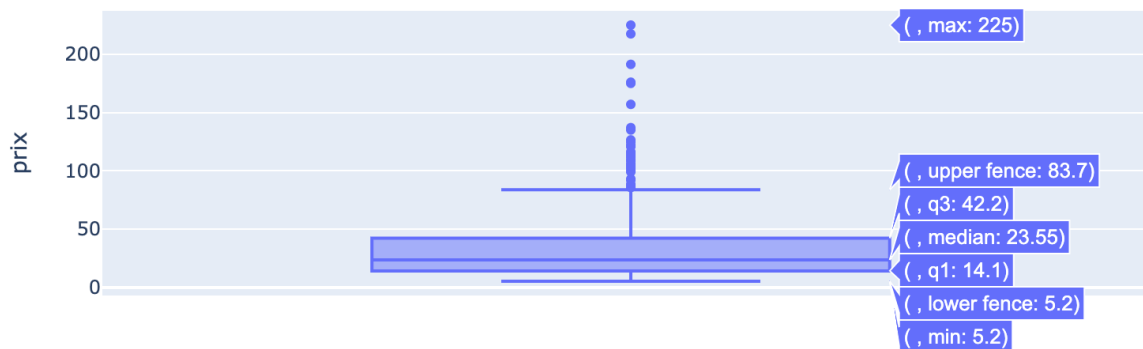
► Importation librairie plotly

```
[45]: import plotly.express as px
```

```
[46]: fig = px.box(df_final, y="price", points="outliers", hover_data=["product_id", "CA_produit"],  
                title="Valeurs aberrantes sur le prix")  
fig.update_layout(yaxis_title="prix")  
fig.show()
```



Valeurs aberrantes sur le prix



11. Détection des outliers

11.4. Méthode du z_score

```
[47]: moyenne = df_final["price"].mean()
      std = df_final["price"].std()
```

```
[48]: Valeurs_aberrantes_2 = df_final[(df_final["price"] - moyenne).abs() >= 3*std]

print("Liste des prix aberrants :")
print(Valeurs_aberrantes_2["price"].values)

pourcentage_Valeurs_aberrantes_2 = round(len(Valeurs_aberrantes_2) / len(df_final) * 100, 2)

print(f"Le nombre de valeurs manquantes est : {len(Valeurs_aberrantes_2)}, soit {pourcentage_Valeurs_aberrantes_2}%")
```

Liste des prix aberrants :

```
[225.  126.5 176.  157.  137.  217.5 124.8 175.  191.3 122.  135.  116.4
 121.  121. ]
```

Le nombre de valeurs manquantes est :14, soit 1.96%

11. Détection des outliers

11.5. Affichage des outliers

```
[73]: outliers= set(Valeurs_aberrantes["price"].tolist() + Valeurs_aberrantes_2["price"].tolist())  
      print(len(outliers))  
      print(np.array(sorted(outliers)))
```

29

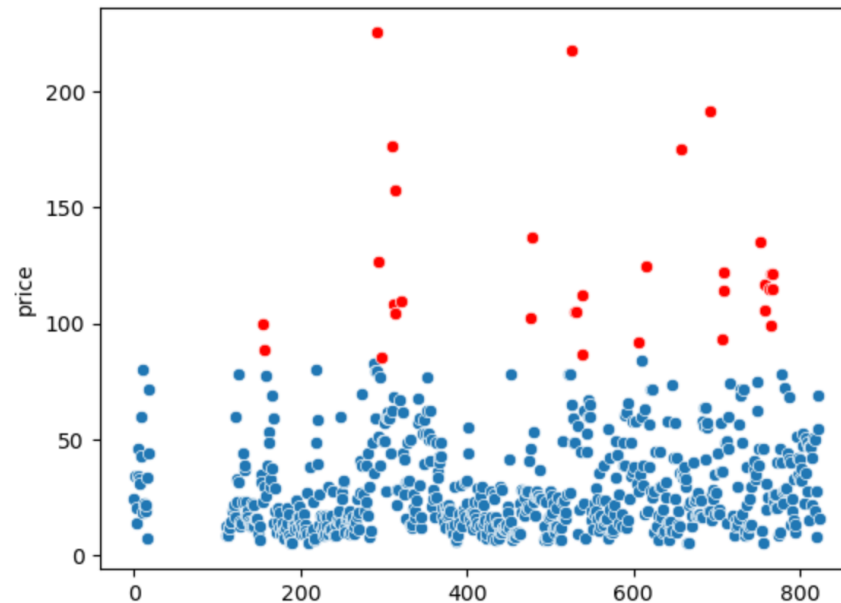
```
[ 85.6  86.8  88.4  92.   93.   99.  100.  102.3 104.  105.  105.6 108.5  
 109.6 112.  114.  115.  116.4 121.  122.  124.8 126.5 135.  137.  157.  
 175.  176.  191.3 217.5 225. ]
```


11. Détection des outliers

11.6. Création des nuages de points

```
[71]: import seaborn as sns
import matplotlib.pyplot as plt
sns.scatterplot(df_final[df_final["is_outlier_z"]== False],
               x=df_final[df_final["is_outlier_z"]== False].index,
               y="price")

sns.scatterplot(df_final[df_final["is_outlier_z"]== True],
               x=df_final[df_final["is_outlier_z"]== True].index,
               y="price", color= "red")
plt.show()
```



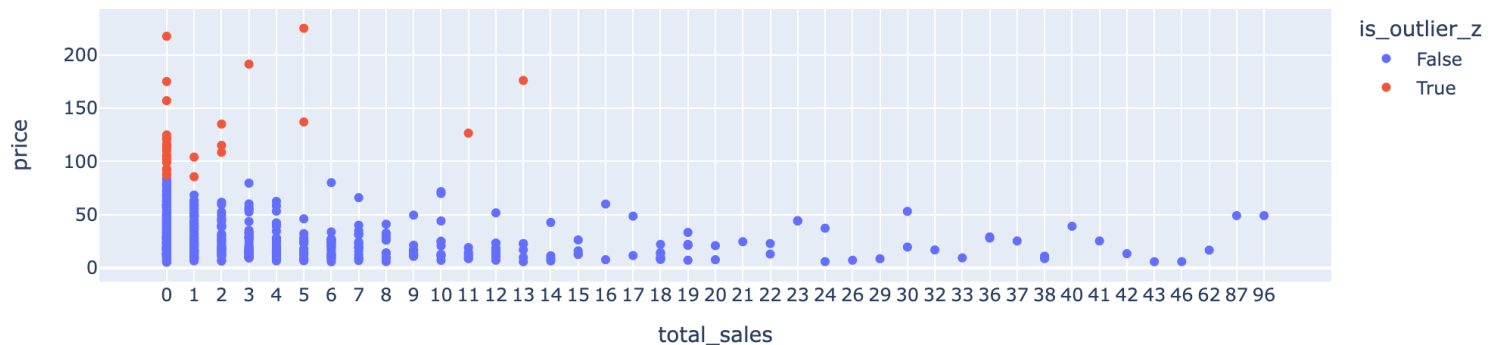
11. Détection des outliers

11.6. Création des nuages de points

```
[57]: df_final["is_outlier_z"] = df_final["price"].isin(list(outliers))

fig = px.scatter(df_final.sort_values(by="total_sales"),
                 x="total_sales",
                 y="price",
                 color="is_outlier_z",
                 hover_data=["price", "CA_produit"],
                 title="Scatter plot des prix avec CA et outliers")
fig.update_xaxes(type="category")
fig.show()
```

Scatter plot des prix avec CA et outliers



12. Conclusion

- ▶ La consolidation des données ERP, Web et table de liaison a permis de créer une base produits fiable et unifiée, facile d'analyse.
- ▶ Les clés produits ont été vérifiées et les doublons ont été traités pour assurer l'unicité.
- ▶ Le choix des jointures extérieures des fichiers a été fait pour recenser l'ensemble des produits.
- ▶ Le chiffre d'affaires a été calculé par produit et en total ligne afin de vérifier.
- ▶ 4,48 % des prix produits ont été identifiés comme valeurs aberrantes par la méthode des interquartiles
- ▶ 1,96 % par la méthode du Z-score
- ▶ Cette différence est due au seuil de 3 pris pour la méthode des Z-score.
- ▶ Il n'y a pas vraiment de valeurs aberrantes. Ici, se sont les prix des vins onéreux qui sont mis en exergue.